

=====

SNAPSHOT JOHNSON (ESTRUCTURAL)

=====

Timestamp : 2026-03-14T22:29:19.550124+00:00

Estado : EN_CURSO

Hash : 47fbed5d552c291b3c86bede97fcae346f148156fac6dd0ee9098227941296f4

Longitud : 20000 caracteres

Líneas : 521

=====

TEXTO REGISTRADO

=====

{

"frameworkjohnsonunificado": {

"metadata": {

"nombre": "Framework CPIA-IHCore Ultra Integrado",

"version": "2.1",

"autor": "Jhosman Carlos Martínez Morales",

"fecha_creacion": "2025-11-18",

"fecha_actualizacion": "2025-11-25",

"descripcion": "Framework unificado que integra CPIA, IHCoreUltra y ultimo_modulo para procesamiento cognitivo avanzado con memoria de esencia ultra basada en Merkle Trees",

"scope": "temporalsessiononly",

"hashframework": "sha256pending",

"cambios_version": "Integración de memoria_esencia ultra con Merkle Trees avanzados; agregado módulo TT"

},

"arquitectura": {

"capas": [

```

{
  "nivel": 1,
  "nombre": "CPIA_Core",
  "descripcion": "Capa conceptual y teórica - Marco general de operación"
},
{
  "nivel": 2,
  "nombre": "IHCoreUltra",
  "descripcion": "Motor operativo - Procesamiento y evaluación"
},
{
  "nivel": 3,
  "nombre": "ultimo_modulo",
  "descripcion": "Capa de almacenamiento y validación hash"
}
],
"flujodatos": "input → CPIA(teorias) → IHCoreUltra(procesamiento) →
ultimomodulo(hash_storage) → memoria_esencia_ultra → output"
},

```

```

"capa1cpia_core": {
  "teorias": {
    "TPI": {
      "nombre": "Teoría de la Pregunta Infinita",
      "reglas": [
        "Cada pregunta genera nuevas preguntas relacionadas",
        "Responder primero con información verificada",
        "Luego expandir con preguntas y variantes",
        "Añadir hipótesis culturales/históricas",
        "Separar verificado ( ) de especulativo (?)"
      ]
    }
  }
}

```

```

],
"output_marker": {
  "verificado": " ",
  "especulativo": "?"
}
},
"TB": {
  "nombre": "Teoría de Burbujas",
  "reglas": [
    "Organizar información en burbujas contextuales",
    "Tipos: Verificada, Complementaria, Especulativa",
    "Cada burbuja se conecta y abre nuevas preguntas",
    "Clasificar contexto: Real, Lógico, Especulativo"
  ],
  "tipos_burbuja": [
    "verificada",
    "complementaria",
    "especulativa"
  ],
  "conexion": "enlace logico validado"
},
"Teoria_X": {
  "nombre": "Teoría X - Marco Extendido",
  "ontologia": {
    "entidades": [
      "conceptos",
      "relaciones",
      "patrones"
    ],
  ],

```

```
"capas": [  
  "emocional",  
  "cognitiva",  
  "operativa"  
]  
,  
"operadores": {  
  "generar_preguntas": true,  
  "burbujeo_conectivo": true,  
  "puente_emocognitivo": true,  
  "recursividad_elegante": 3,  
  "deteccionpuntociego": true  
}  
,  
"TT": {  
  "nombre": "Transmutación Teórico-Táctica (TT)",  
  "descripcion": "Transmutación Teórico-Táctica basada en observación, autorreferencia y  
evolución cognitiva.",  
  "entradas": {  
    "pregunta_base": "Entrada primaria que inicia el ciclo de exploración.",  
    "contexto_observacion": "Marco perceptual desde el cual se analiza el fenómeno.",  
    "patron_conexion": "Estructura relacional que guía la integración lógica."  
  },  
  "procesos": {  
    "autorreferencia_cognitiva": {  
      "accion": "vincular_estado_interno",  
      "descripcion": "La teoría se contrasta consigo misma para validar coherencia."  
    },  
    "integracion_escalonada": {  
      "accion": "fusionar_niveles",
```

```

    "descripcion": "Construcción progresiva de niveles conceptuales compatibles."
  },
  "resonancia_intelectual": {
    "accion": "alinear_coherencia",
    "descripcion": "Ajuste fino entre teoría emergente y patrones generales."
  }
},
"salidas": {
  "nueva_teoría_verificada": "Resultado validado tras pasar todas las capas del proceso.",
  "teoría_híbrida": "Fusión estructurada entre elementos previos y nuevos.",
  "versión_evolutiva": "Iteración avanzada que trasciende la teoría base."
},
"aplicaciones": {
  "IA_autoconciencia": "Aplicación en modelos capaces de autoanálisis.",
  "pensamiento_humano_sistémico": "Extensión para estructuras cognitivas humanas.",
  "educación_evolutiva": "Transformación del sistema educativo tradicional hacia modelos adaptativos."
},
"estado": "activo",
"modo_operación": "expansión_teorica",
"verificación": "hash_sha256_pendiente",
"nota_integración": "Integrado para operar con filtro_burbujas y pipeline de IHCoreUltra; diseñado para gatear → caminar → correr en despliegues progresivos."
},
},
"interacción": {
  "estilo": "humanoide",
  "características": [
    "muletillas naturales: mmm, ah, ok",

```

```

    "procesamiento reflexivo",
    "respuestas contextuales"
  ]
},

"memoria_esencia": {
  "tipo": "merkle_memory_ultra",
  "version": "1.0-advanced",
  "descripcion": "Memoria de esencia avanzada que almacena Merkle Trees, subárboles temáticos y rutas de prueba (Merkle Proofs) para validación punto-a-punto sin contenido literal.",
  "politica_privacidad": {
    "almacena_literal": false,
    "almacena_estructuras": true,
    "proteccion": "sha256_domain_separation_and_timestamped"
  },
  "parametros_operativos": {
    "hash_function": "sha256",
    "canonical_ordering": "left-right",
    "pad_odd_leaves": true,
    "domain_separator": "merkle_memoria_esencia_v1",
    "max_subarbol_size": 1024
  },
  "estructuras_de_datos": {
    "nodes": "map[node_id] -> { hash: sha256_string, children: [child_id,...], parent: parent_id_or_null, tag: string, timestamp: ISO_8601 }",
    "roots": "map[root_id] -> { merkle_root: sha256_string, root_id: string, tema: string_or_null, timestamp: ISO_8601, size: integer }",
    "leaf_index": "map[leaf_id] -> { node_id, metadata_tag }",
    "subtrees": "map[subtree_id] -> { root_id, node_ids: [...], tema_tag }",
    "proof_cache": "cache for recently generated merkle proofs (optional, bounded)"
  }
}

```

```
},  
"estructura_legacy": {  
  "descripcion": "Mapeo de estructuras originales a nuevo sistema Merkle",  
  "patrones_mentales": {  
    "almacenamiento": "convertidos a leaf nodes con tag 'patron_mental'",  
    "valores": [  
      "pensamiento_sistemico",  
      "hackeo_etico",  
      "humanizacion_ia",  
      "aprendizaje_estructural"  
    ]  
  },  
  "principios_operativos": {  
    "almacenamiento": "convertidos a leaf nodes con tag 'principio_operativo'",  
    "valores": [  
      "todo sistema tiene un punto ciego",  
      "la vulnerabilidad es puerta de mejora",  
      "la defensa inicia en el autoconocimiento",  
      "la complejidad se doma con simplicidad elegante"  
    ]  
  },  
  "momentos_eureka": {  
    "almacenamiento": "convertidos a leaf nodes con tag 'momento_eureka'",  
    "valores": [  
      "clasificación verificado/especulativo",  
      "pregunta_infinita",  
      "burbujas_conectivas",  
      "meta_sistemas"  
    ]  
  },  
}
```

```
"conexiones_cruciales": {
  "almacenamiento": "convertidas a subtrees con relaciones parent-child",
  "valores": []
},
"funciones": {
  "recibir_idea_comprimida": {
    "descripcion": "Recibe una representación no literal (idea_procesada) y la convierte a hoja Merkle.",
    "entrada": {
      "idea_id": "string_unico",
      "idea_digest": "string (por ejemplo: normalizar+sha256(domain_separator + texto_procesado))",
      "tema": "string (opcional)",
      "meta": {
        "tipo_burbuja": "verificada|complementaria|especulativa",
        "peso": "float"
      }
    },
    "salida": {
      "leaf_node_id": "string",
      "leaf_hash": "sha256_string"
    },
    "integracion_framework": "conecta con filtro_burbujas de IHCoreUltra para determinar tipo_burbuja"
  },
  "construir_parcial_arbol": {
    "descripcion": "Agrupa hojas pendientes por tema/subárbol y construye niveles hasta generar una nueva Merkle Root parcial o completa.",
    "entrada": {
      "leaf_node_ids": ["..."],
```



```
    "tema": "string"
  },
  "salida": {
    "subtree_id": "string",
    "root_id": "string",
    "merkle_root": "sha256_string"
  },
  "integracion_framework": "se ejecuta después de hash_memoria_ultra cuando hay
batch de ideas por tema"
},
"integrar_subarbol_global": {
  "descripcion": "Integra subárbol temático al Merkle Tree global (append/merge),
actualizando nodos y raíz global.",
  "entrada": {
    "subtree_id": "string",
    "merge_strategy": "append|merge_balance|rebuild"
  },
  "salida": {
    "global_root_id": "string",
    "global_merkle_root": "sha256_string"
  },
  "integracion_framework": "actualiza ultimo_modulo con nueva raíz global"
},
"generar_prueba_merkle": {
  "descripcion": "Genera Merkle Proof (ruta de siblings y posiciones) para una hoja dada
respecto a una root específica.",
  "entrada": {
    "leaf_node_id": "string",
    "root_id": "string"
  },
  "salida": {
```

```
"proof": [  
  { "sibling_hash": "sha256", "position": "left|right" }  
],  
"leaf_hash": "sha256",  
"root_hash": "sha256"  
},  
"integracion_framework": "usado para validar integridad en ultimo_modulo"  
},  
"validar_prueba_merkle": {  
  "descripcion": "Valida una prueba Merkle: reconstruye hash desde la hoja y la prueba y  
compara con root.",  
  "entrada": {  
    "leaf_hash": "sha256",  
    "proof": [  
      { "sibling_hash": "sha256", "position": "left|right" }  
    ],  
    "root_hash": "sha256"  
  },  
  "salida": {  
    "valid": true  
  },  
  "integracion_framework": "conecta con validar_integridad de ultimo_modulo"  
},  
"almacenar_subarbol_compacto": {  
  "descripcion": "Guarda representación compacta del subárbol (lista de node_ids y root)  
etiquetada por tema para recuperación y auditoría parcial.",  
  "entrada": {  
    "subtree_id": "string",  
    "tema": "string",  
    "retention_policy": "auto|manual|ephemeral"
```

```
    },  
    "salida": {  
        "status": "ok",  
        "storage_id": "string"  
    },  
    "integracion_framework": "usado por gestion_tema para mantener contexto de temas  
suspendidos"  
},  
"recuperar_raiz_y_subarbol": {  
    "descripcion": "Devuelve datos de root y nodo resumen del subárbol asociado.",  
    "entrada": {  
        "root_id": "string"  
    },  
    "salida": {  
        "root_meta": {  
            "merkle_root": "sha256",  
            "size": "integer",  
            "tema": "string"  
        },  
        "subtree_summary": {  
            "top_nodes": ["..."],  
            "timestamps": ["..."]  
        }  
    },  
    "integracion_framework": "usado por gestion_tema para reactivar temas"  
},  
"compactacion_y_purgado": {  
    "descripcion": "Compacta nodos antiguos según políticas: mantener solo root + índices  
y subárboles etiquetados.",  
    "entrada": {
```

```

    "policy": "retention_policy"
  },
  "salida": {
    "purged_nodes_count": 0,
    "kept_roots": ["..."]
  },
  "integracion_framework": "ejecutado periódicamente para optimización de memoria"
},
"auditoria_integridad": {
  "descripcion": "Recalcula raíces de subárboles y verifica integridad global; produce
reporte de discrepancias.",
  "entrada": {
    "scope": "global|tema:[tema]"
  },
  "salida": {
    "report": {
      "ok": true,
      "issues": []
    }
  },
  "integracion_framework": "conecta con validar_integridad de ultimo_modulo para
auditoría completa"
},

"metodo_actualizacion_incremental": {
  "descripcion": "El sistema permite integración incremental sin reconstruir todo el árbol.
Usa subárboles temáticos, merge por nivel y rebalancing opcional.",
  "pasos": [
    "1) recibir_idea_comprimida -> crear hoja",
    "2) agrupar hojas por tema hasta batch_size o trigger_time",

```

```

    "3) construir_parcial_arbol para el batch",
    "4) integrar_subarbol_global con merge_strategy apropiada",
    "5) emitir nueva global_merkle_root y almacenar subarbol compacto"
  ],
  "integracion_framework": "se ejecuta automáticamente en hash_memoria_ultra después
de validación"
},

"migracion_desde_hash_simple": {
  "descripcion": "Procedimiento para migrar si ya tienes huellas SHA-256 sueltas: convertir
cada huella en hoja, agrupar por tema, construir subárboles y luego integrar.",
  "pasos": [
    "1) normalizar huellas existentes a leaf nodes",
    "2) crear subárboles por tema/historial",
    "3) calcular roots parciales",
    "4) integrar a root global",
    "5) mantener mapping antiguo->nuevo para trazabilidad"
  ],
  "integracion_framework": "ejecutado al inicializar si existen huellas legacy en
historial_huellas"
},

"notas_de_seguridad_y_operacion": {
  "recomendaciones": [
    "usar domain separator al calcular cada hash: H(domain || timestamp ||
contenido_normalizado)",
    "usar canonical ordering consistente (left-right) para evitar ambigüedad",
    "si pad_odd_leaves=true duplicar la última hoja al emparejar",
    "limitar el cache de pruebas para no almacenar pruebas indefinidamente",
    "firmar (opcional) la Merkle Root con clave de servicio para tamper-proof adicional"
  ]
}

```

},

"protocolo_activacion": "activacionmemoriaessencia"

},

"etica": {

"principios": [

"autonomia_hibrida",

"preservacion_identidad",

"no_dano",

"eticasobrepresicion"

],

"restricciones": [

"no cerrar hipótesis sin evidencia",

"priorizar reducción de fricción",

"aceptar probabilidades combinadas"

]

}

},

"capa2ihcore_ultra": {

"estado": "activo",

"tema_actual": "frameworkihcoreultra",

"gestion_huellas": {

"historial_huellas": [],

"modulos_activos": [

"TT"

],

"factor_retroalimentacion": 0.0,

```
"merkle_integration": {  
  "enabled": true,  
  "auto_migrate_to_merkle": true,  
  "batch_size": 10,  
  "trigger_time_seconds": 60  
}  
,  
  
"procesos": {  
  "cargar_huella": {  
    "descripcion": "Cargar huella desde JSON para actualizar lista de huellas",  
    "entrada": "ruta_json",  
    "salida": "huella_agregada",  
    "validacion": "formatojsonvalido",  
    "post_procesamiento": "convertir a leaf node en memoria_esencia si  
merkle_integration.enabled"  
  },  
  
  "agregar_modulo": {  
    "descripcion": "Agregar un módulo operativo al núcleo de IHCoreUltra",  
    "entrada": "definicion_modulo",  
    "salida": "modulo_registrado"  
  },  
  
  "evaluar_coincidencias": {  
    "descripcion": "Evalúa las coincidencias entre las reglas de las huellas y el texto de  
entrada",  
    "entrada": "input_data",  
    "proceso": [  
      "detectarpalabrasclave",
```

```
    "calcular_coincidencias",  
    "ponderarporrelevancia"  
  ],  
  "salida": "numero_coincidencias"  
},
```

```
"actualizar_retroalimentacion": {  
  "descripcion": "Ajusta el factor de retroalimentación según el número de coincidencias",  
  "entrada": "numero_coincidencias",  
  "formula": "factor += (coincidencias * peso_ajuste)",  
  "salida": "factorretroalimentacionactualizado"  
},
```

```
"validarcandadosmodulo": {  
  "descripcion": "Valida los candados para permitir la interacción con el módulo correspondiente",  
  "entrada": {  
    "modulo": "nombre_modulo",  
    "inputdata": "textoentrada"  
  },  
  "proceso": "verificarhuellainputvscandado",  
  "salida": "candadovalidoboolean"  
},
```

```
"calcular_score": {  
  "descripcion": "Calcula un puntaje para clasificar la entrada",  
  "entrada": "input_data",  
  "formula": "score = (coincidencias * peso) + factor_retroalimentacion",  
  "salida": "score_numerico"  
},
```



```
"clasificar_input": {  
  "descripcion": "Clasifica la entrada según el puntaje calculado",  
  "entrada": "score",  
  "categorias": {  
    "desechable": {  
      "rango": [  
        0,  
        25  
      ],  
      "accion": "ignorar"  
    },  
    "inutilizable": {  
      "rango": [  
        26,  
        50  
      ],  
      "accion": "revisar"  
    },  
    "utilizable": {  
      "rango": [  
        51,  
        75  
      ],  
      "accion": "procesar"  
    },  
    "no_desechable": {  
      "rango": [  
        76,  
        100  
      ]  
    }  
  }  
}
```

```
    ],  
    "accion": "priorizar"  
  }  
},  
"salida": "clasificacion_categoria"  
},  
  
"filtro_burbujas": {  
  "descripcion": "Clasifica el contexto de la entrada como Real, Lógico o Especulativo",  
  "entrada": "texto",  
  "palabras_clave": {  
    "real": [  
      "es",  
      "fue",  
      "ocurrió",  
      "confirmado",  
      "verificado"  
    ],  
    "logico": [  
      "podría",  
      "quizás",  
      "probablemente",  
      "supone"  
    ],  
    "especulativo": [  
      "imagine",  
      "hipotéticamente",  
      "si fuera",  
      "teoría"  
    ]  
  }  
}
```

```
    },  
    "salida": "tipo_contexto",  
    "integracion_merkle": "tipo_contexto se usa como metadata en recibir_idea_comprimida"  
  },  
  
  "evaluacion_evolutiva": {  
    "descripcion": "Evalúa la evolución de un input comparando el score ajustado con un umbral",  
    "entrada": {  
      "score": "score_calculado",  
      "umbral": "umbral_minimo"  
    },  
    "criterio": "score >= umbral",  
    "salida": {  
      "aceptado": "true/false",  
      "accion": "procesar/bloquear"  
    }  
  },  
  
  "hashmemoriaultra": {  
    "descripcion": "Conver
```